

A Multi-Modality Visualizer Interface for Intelligent Image Processing

Final Group Report

Group15

Yuhao Guan, Hao You, Pengzhou Xu

ECE532: Digital System Design
Spring 2025
Faculty of Applied Science and Engineering
University of Toronto

Table of Contents

1	Overview.....	3
1.1	Introduction	3
1.2	Project Description.....	3
1.3	Block Diagram	3
1.4	IP overview.....	4
1.5	Project Complexity.....	5
2	Outcome.....	5
2.1	Result.....	5
2.2	Future Work	6
3	Project Schedule	6
3.1.1	Discussion:	7
4	Description of the Blocks.....	8
4.1	Camera Unit.....	8
4.1.1	Camera control IP.....	8
4.1.2	Camera capture IP.....	9
4.2	BRAM Unit	10
4.3	Image Processing Unit.....	11
4.3.1	Convolutional Processing Unit.....	11
4.3.2	Grey-scale	11
4.4	VGA Interface Unit.....	12
4.4.1	VGA output IP	12
4.4.2	GUI.....	13
4.5	Keyboard Interface Unit.....	14
5	Description of the Design Tree	15
6	Tips and Tricks	16
6.1	Handling clock domain crossing (CDC)	16
6.2	Reduce OV7670 noise.....	16
7	Video	16
8	Reference	17

1 Overview

This section introduced motivation and basic information about the project. Block diagram, IP overview, and project complexity table are also provided to give a clear understanding of the system architecture and design scope.

1.1 Introduction

Intelligent image processing has emerged as a cornerstone of artificial intelligence (AI) in recent years. Since the last century, image processing algorithms, such as JPEG compression and wavelet transforms, have provided efficient methods for image compression. Among these advancements, convolutional neural networks (CNNs) have revolutionized how machines perform important tasks such as image recognition. This technology has enabled applications like autonomous driving, facial recognition, and more broadly, AI computation that redefines modern technology.

This project aims to develop an interactive educational platform for visualizing image-processing tasks. These operations are essentially multiply-and-accumulate (MAC) computations, which are often abstract and difficult to grasp. Therefore, we aim to develop a platform that allows users to explore various image transformations and intuitively visualize the results. The system will contain multiple modalities, enabling users to toggle among image transformations from camera input. The task will be performed with a Xilinx DDR FPGA and peripheral modules (Pmod) including an OV7670 camera, VGA display and a keyboard.

1.2 Project Description

The project is an intuitive interface that accepts users' selection of filter and display mode, then displays the frame of image captured or video stream in real time. Users interact with the system by pressing certain keys on a keyboard to toggle between different filters and use a switch on the FPGA to change modes. The FPGA is programmed to apply various image-processing operations to each image frame based on users' selections. The processes include one pixel-wise processing which is the Greyscale filter, and two convolutional-based operations which are Gaussian blur and Sobel filter. The transformed image is displayed via VGA display using a single frame buffer, with a user-friendly graphical interface showing the graphical filter button selection and the filter's name as title. This project integrates mostly hardware custom IPs and some Xilinx IPs to implement system functions, emphasizing visualization to serve as an interactive and educational platform for exploring image processing techniques.

1.3 Block Diagram

Fig. 1 illustrates the block diagram of our system containing the main IP blocks and their interconnections. The dataflow starts when the Camera controller tells the OV7670 Camera PMOD to capture a frame of image data. This data is either directly stored in Xilinx BRAM or first processed by the Convolutional Image Processor depends on users' selections. For the output dataflow, the VGA Display controller reads the image data from the BRAM. Depending on users' choices, this data may be filtered by the Grey-scale Image Processor. In either case, each pixel value arrives is sent to the physical VGA Display PMOD, except that some pixels will be replaced by the GUI elements as the final output. Finally, regarding the control flow, a Keyboard Interface IP

receives the keyboard signals sent when keys are being pressed. The IP then translates it into filter selection signals and sends them to the rest of the system where they are needed.

Furthermore, all the white blocks are custom IPs, and the only Xilinx IP is BRAM. All the interface between the blocks are point-to-point communication and the validation of the data transfers and receives is promised by delicately designed FSMs in different blocks.

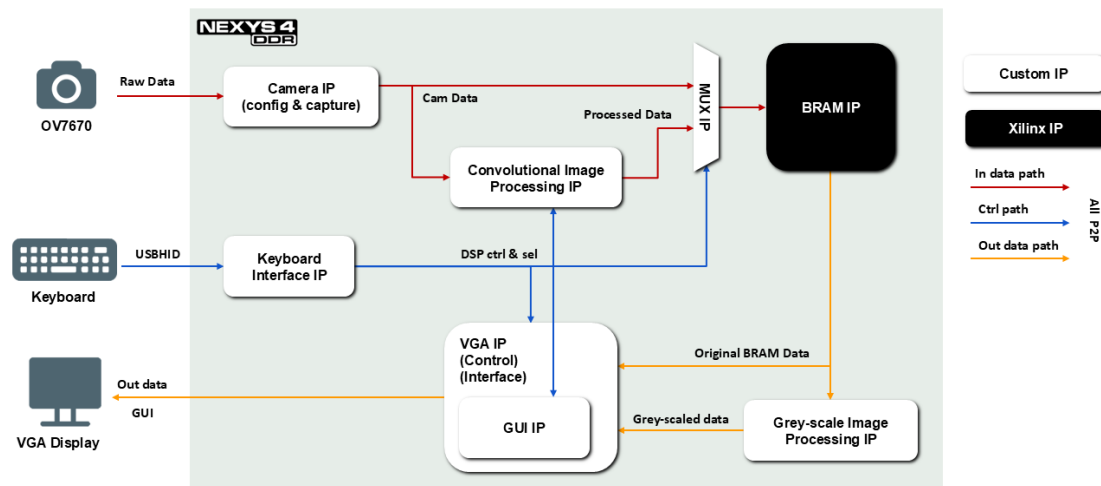


Fig. 1 Block Diagram of the System.

1.4 IP overview

All the IP usage and their brief description are provided in Table 1. More details regarding each of them are provided in the Block Description Section of this report.

Table 1 Summary of IP used in the system.

IP Name	Description
Camera IP	Configures the physical camera pmod and communicates with the camera with I2C to constantly capture frames of images.
BRAM IP	Xilinx BRAM works in simple Dual port mode. In all time, it stores a size of one frame image.
Convolutional Image Processing IP	Performs a convolutional calculation on the image with 3x3 kernels for Gaussian blur and Sobel filter.
Grey-scale Image Processing IP	Performs a pixel-wise normalization to transform the RGB data to their corresponding Grey-scale data
VGA IP	Iteratively reads from the image pixel addresses following the screen position order, then does extra pixel-wise processing to each pixel before sending to VGA display pmod.
GUI IP	Embedded inside the VGA IP but individually stores and updates the pattern of GUI.
Keyboard Interface IP	Communicates with the keyboard pmod with PS2 and translates the received keyboard scan code into the control signals.

1.5 Project Complexity

The complexity of the project is summarized in Table 2 below.

Table 2 Summary of the project complexity score

Complexity	Score
OV7670 VGA Camera	1.0
VGA output (without MicroBlaze)	1.0
Custom Image Processing IP Core	1.0
Visualize meaningful results with a GUI	0.75
USB keyboard implementation	0.5
Total	4.25

2 Outcome

2.1 Result

Functionally wise, the project completed 100% of the originally proposed goals. This includes real-time video capture by the camera, frame-by-frame processing by the image processor, and real-time display of the processed frames through the VGA display. Meanwhile, users can press keyboard keys to switch between different filters, and the video stream and GUI elements respond accordingly also in real time.

A sequence of images demonstrating this functionality is provided below (Fig. 2-5). The images are 4 different video frames from a VGA display, and it is displaying a user pressing different keys on the keyboard except the frame styles and GUI elements were updated. This video displayed on the VGA display is captured by OV7076 Camera. To better see video updates except for filter styles, one can focus on the user's finger and GUI elements (i.e. buttons and title).

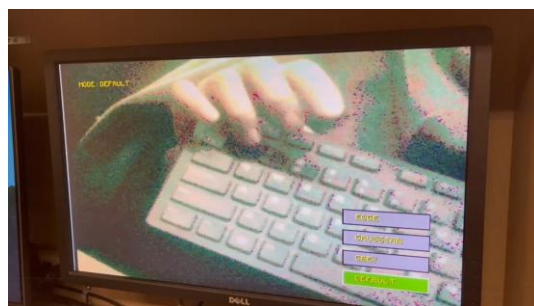


Fig. 2 Default mode display output



Fig. 3 Grey-scale display output

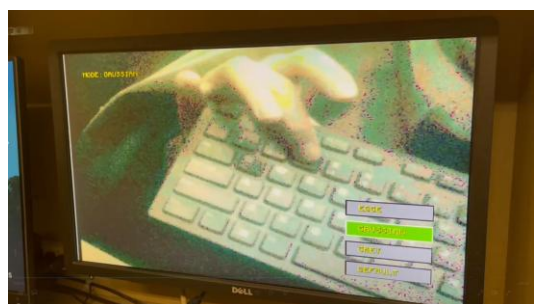


Fig. 4 Gaussian blur display output



Fig. 5 Sobel filter display output

2.2 Future Work

Technology wise, due to one team member dropping the course, the team was unable to successfully implement all the originally planned technologies, including the DDR3 Memory and the Video Direct Memory Access (VDMA) IP. The DDR3 Memory was intended to provide the system with additional storage capacity to store 2 frames of video, which will enable the VGA display to work in double buffer mode. Under this mode, the system will be able to processes and output one frame stored while the camera simultaneously captures the next frame. This will greatly improve the video frame rate. The VDMA IP was planned to be used to stream the frames camera captured directly to one frame location in the DDR3 Memory while bypassing the CPU, which will further improve the frame rate.

The team made efforts to improve the design with the above technologies but later have to abandon them due to the reduced workforce. If offering more time or someone were to continue the project, these components should be prioritized for implementation.

3 Project Schedule

Table 3 Comparison between the Original Milestone and the Current Milestone

Original Milestone	Current Milestone
Milestone 1	
● Do preliminary research on tasks to gain an overview of the design requirements ● Research on optimizations, algorithms, or protocols of peripherals	● Research keyboard protocol ● Research OV7670 camera data feed logic ● Research convolutional image processing methods ● Research VGA control logic
Milestone 2	
● Test peripherals' functionality ● Refine implementation plan with further detail and testing procedure ● Focus on getting inputs from the keyboard and decoding the pressed key	● Develop matrix multiplication logic in convolution IP with RGB input pixels ● Set up the I2C for OV7670 camera ● Design a first version integrated block diagram ● Connect camera to FPGA for testing ● Implement the USB keyboard with PS/2 protocol
Milestone 3	
● realize the functionalities of each building block ● Aim to get images on the camera and get a better insight into data processing	● Package stream to VGA IP ● Configure DMA ● Store camera capture data in a BRAM block ● Show BRAM camera capture frame to the VGA with delay and synchronization ● Attempt to build a double buffer with two stacks of BRAM ● Visualize GUI design ● Pack elements to Verilog arrays ● Display testing characters on VGA.

Milestone 4	
● Realize displays and GUI functions from the captured images. ● Pass test benches for each sub-module.	● Design block diagram with MicroBlaze for integration ● Integrate the camera with VDMA and debug ● Redo camera capture and control IP to integrate with MicroBlaze. ● Integrate frame data with GUI arrays ● Lower the memory usage of the GUI through Python tool refining ● Connect the switch control signal to the VGA to replace keyboard for testing
Milestone 5	
● Assemble each part of the design ● Create customized IPs to integrate all functions for final presentations ● Realize additional modalities, filters, or a more intuitive GUI design	● Complete the convolution IP and greyscale IP interface ● Simulate the convolution and greyscale IP ● Modify the camera capture IP to support both video and capture modes ● Update the camera capture IP to use AXI stream output ● Provide sync signals to the image processing IP ● Design control signals to select different filters ● Try to integrate convolution IP into frame capture system
Milestone 6	
● focus on drafting presentations ● Finalize reports ● Preparing for demo and presentations. ●	● Modified the filter communication interface to enable its connection with the camera and BRAM ● Debug convolutional IP integration ● Add a new convolutional filter ● Compress the captured data ● Establish the selection structure between the camera capture and filter

3.1.1 Discussion:

Generally, the current milestone is much more complex than what we expected in the beginning of the course, due to the fact that we did not come up with a robust design plan at first. Many modules have substantial amount of work and debugging requirement, thus we cannot meet the high pace set in the beginning. Although the current milestone has more contents every week, the first three milestones are similar with the original plan, since it is just at the starting stage of the project, everything went well at the beginning. After the third milestone, the team undergoes some technical difficulties as well as change in group members. Consequently, the original milestone does not provide accurate information of the project progress, but it still serves as a general checklist about general work that is left.

The greatest change occurred at the time just after the mid-term demo. One of the team members dropped the course and we decided to reduce the scope for the project. Many of the team members spent significant amount of time in configuring the VDMA and interfacing it with the main project,

but we opted for dropping the VDMA module and move on with the frame capture version of the project. In this case, we must reconsider how to set up the main project diagram and how to synchronize each modules/IPs. Ultimately, we did not have enough time to dump all technical work and spend a whole week preparing the demo, presentations, and report. In a nutshell, the current milestone reveals that the team members are making prominent progress every week, in a different way with the original plan.

4 Description of the Blocks

4.1 Camera Unit

This project employs OV7670 camera, low voltage CMOS image sensor, to accept video stream from outside. It has multiple color modes and streams video at size of 640×480 pixels that corresponds to the VGA output size. Fig. 6 below shows the block diagram that describes how this project interfaces with OV7670 camera.

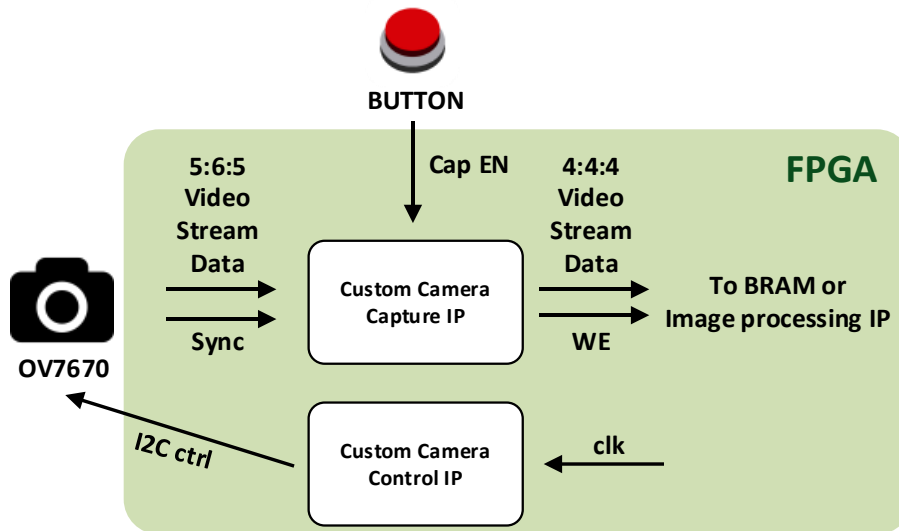


Fig. 6 Block diagram of interfacing between FPGA and OV7670

The interfacing with OV7670 mainly composed of two IP: one for image/video capturing, and one for control. The camera control IP will control OV7670 via I2C protocol. The camera capture IP will accept the 5:6:5 RGB format data from OV7670, compress them to 4:4:4 RGB format, and then send them to either BRAM or image processing IP. Section below will explain the two IP respectively in detail.

4.1.1 Camera control IP

The OV7670 camera module communicates with FPGA in standard SCCB interface which is also compatible with I2C interface. This project controls the OV7670 settings with I2C interface. Fig. 7 below shows an overview of the camera control IP.

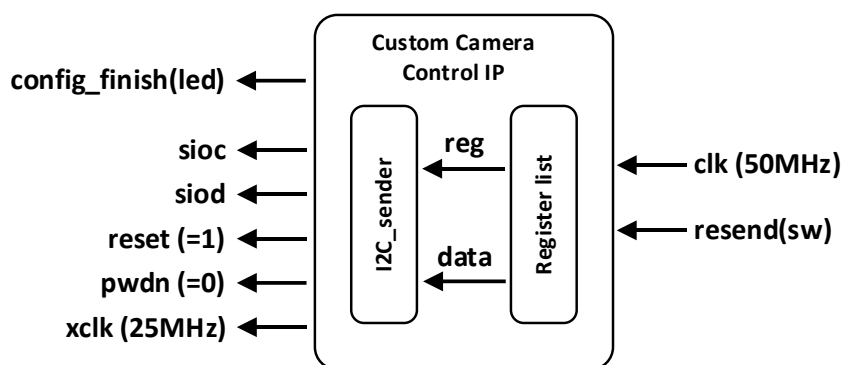


Fig. 7 Custom camera control IP block diagram

The control IP is driven by a 50MHz clock and a resend signal. The resend signal is connected to a physical switch on the FPGA board. It acts as a reset for the control IP. When it asserts 1, the control IP will resend all registers and corresponding values again. Inside the IP, it has a register list and a I2C sender. The I2C sender will accept register address and its data from register list and send them to OV7670 in I2C protocol. The output I2C communication lines are sioc and siod. reset is set to 1 and pwn is set to 0 to allow OV7670 in normal mode and powering up. A 25MHz xclk is provided to drive the camera. When the configuration is finished, config_finish that connect to a led will light up. To better compatible with the VGA output, the OV7670 is configured into 5:6:5 RGB color mode (2 bytes per pixel) to match with the 4:4:4 RGB color mode of VGA easily.

4.1.2 Camera capture IP

The camera capture IP can be either configured into capture mode or slow frame video mode. Fig. 8 below shows an overview of the camera capture IP.

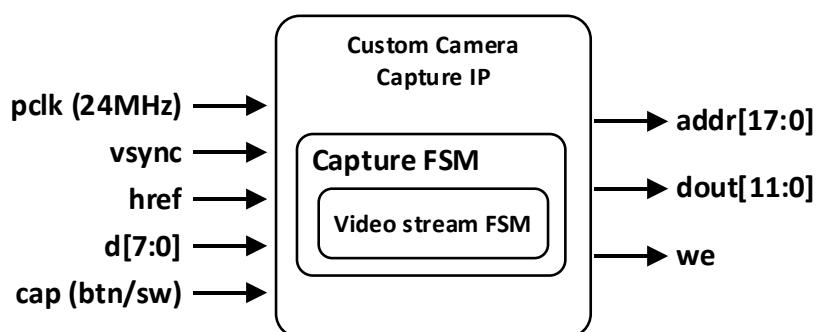


Fig. 8 Custom camera capture IP block diagram

The OV7670 provide the capture module a pclk (about 24MHz) to synchronize with camera data. Two sync signals, vsync and href, are provided for vertical and horizontal synchronization. The video stream data is d[7:0]. Each two cycle of d (2 bytes) will form a complete pixel information in 5:6:5 RGB format. A separate cap signal connected to either button or switch on the FPGA board tell the capture module when to start capture.

Inside the camera capture IP, the video stream FSM will continuously accept video stream data from camera. A capture FSM is nested outside to tell video stream FSM when to stop capture and get only a frame. Fig. 9 below shows the capture mechanism.

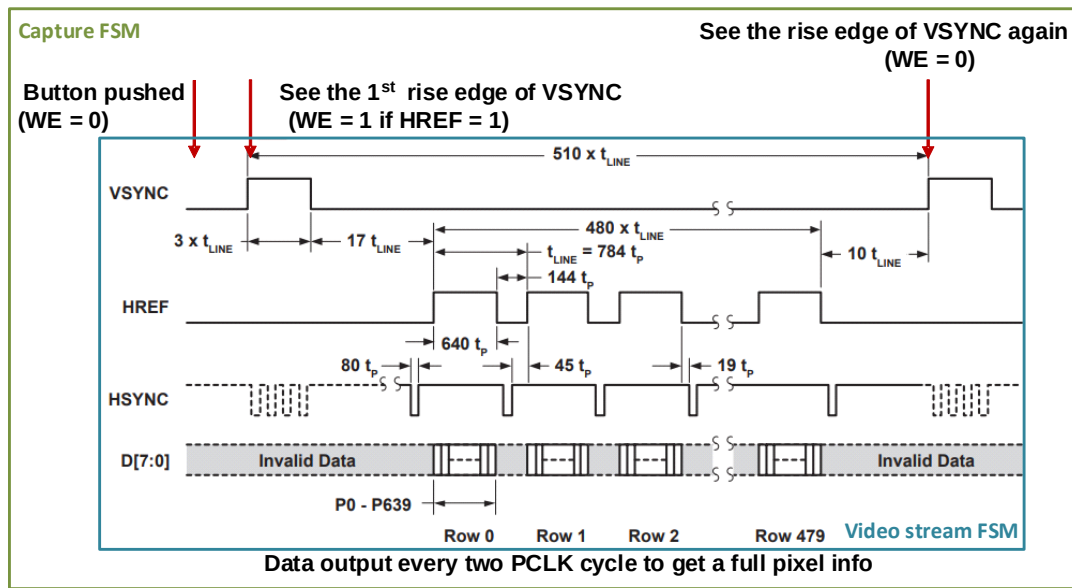


Fig. 9 Video stream FSM and capture FSM [1]

The video stream FSM strictly follows the sync signal from the camera. At the rising edge of the vsync, a new frame will start. Each rising edge of href means a new start of the line. Since each time it only transfers a byte data, it will take two pclk cycle to record a full pixel's information in 5:6:5 RGB format. This data will immediately be truncated into 4:4:4 RGB format and sent out.

The capture FSM is nested outside the video stream FSM. When cap is triggered to 1, we (write enable) will be in wait state but still 0. Once a rising edge of vsync is detected, a new frame has started, and we will be 1 when data is valid. This will continue until the next rising edge vsync where the first frame is end and the second frame has started. we will back to 0. The capture FSM can also be configured into slow frame video, if we can provide a steady periodic cap so that camera can continuously capture in slow frame rate. The fame rate cannot be too high as the output is also connected to image processing IP. Too high frame rate will make the image processing IP cannot process the data immediately and thus make the output out of sync.

The output of the camera capture module will be addr[17:0] (represent the data location), d[7:0] (pixel info in 4:4:4 RGB format), and we (write enable that allow data write to destination and allow other IPs sync with input frame). These outputs will be either stored directly to BRAM, or fetch to image processing IP to apply filter.

4.2 BRAM Unit

We instantiate the BRAM block from the native Vivado IP. As our write clock (camera clock in 24MHz) and read clock (VGA clock in 25MHz) are different, we use simple dual port BRAM where port A is written only for writing data from camera with write enable and port B is read only for VGA reading. Due to the limit number of BRAM inside the board we used (Nexys DDR4, 135 RAM36 in total), the BRAM is set to 12 bits width and 262144 depths. The data from the camera is compressed to half so that the two near pixels used the same color information.

4.3 Image Processing Unit

There are two filter IPs in the whole system. The first is a pixel-wise combinational logic filter that manipulates the color of the frame captured. The second IP contains the major filters that implements convolutional computations. It includes filters such as the Gaussian Blur filter and the Sobel filter.

4.3.1 Convolutional Processing Unit

The convolutional filter IP utilizes several different filtering techniques such as the Gaussian Blur filter and the Sobel filter. The input of this module is the captured frame data coming from the OV7670 camera. Since we only plan to use one dual port BRAM in the project, we do not have enough resources to store the captured frame in the memory first and then extract it back into convolutional filters. The whole structure diagram of this IP is displayed in Fig.10 below.

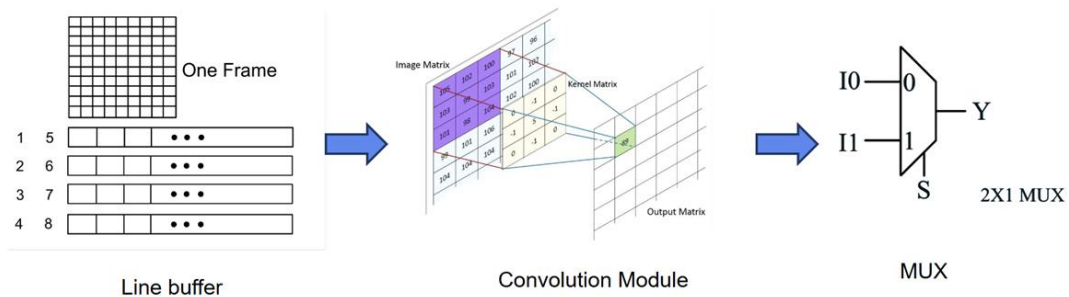


Fig. 10 Convolutional IP block diagram

The incoming RGB444 pixels are stored partially in a line buffer with a maximum of four lines of pixels stored, as the 3x3 convolutional kernel requires 9 pixels from 3 lines of the frame respectively. The extra line in the line buffer provides an extra safety buffer for the data transfer and logic design. The control logic of the line buffers uses Finite State Machines to track which 3 lines are read from the frame for the matrix convolution. After the 9 pixels are carefully selected by the line buffer control logic, they are fed into the next stage where the matrix multiplication is executed. In the matrix multiplication section, two computation algorithms are developed separately for the Gaussian Blur and the Sobel filter. Using keyboard input control signals, one of the filters is selected. The final data from the matrix multiplication is the output of the convolutional filter IP, but it requires some extra address synchronization before being fed into the BRAM. Provided with some synchronization signals from the previous stage and an internal valid signal, the address control logic is designed along with the matrix multiplication logic. Before feeding the pixel into the dual port BRAM, a multiplexer IP is inserted to select between the frame captured directly from the camera and the frame processed.

4.3.2 Grey-scale

The greyscale filter is a color conversion IP that takes input directly from the VGA camera. It does not have internal clock delays, because of its intrinsic property of manipulating single-pixel color data. The weighted sum of the greyscale pixel is encompassed in this formula: Weighted sum: gray

$= (0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B)$. With the color conversion pattern given, the only task is to sum up the weighted RGB input. As a result, this IP doesn't need extra synchronization at all. The output of the color conversion filter is directly connected to the VGA display module.

4.4 VGA Interface Unit

The project displays the captured image/video before and after processing through VGA display. Fig. 11 below shows the block diagram about how the project outputs the data to the VGA.

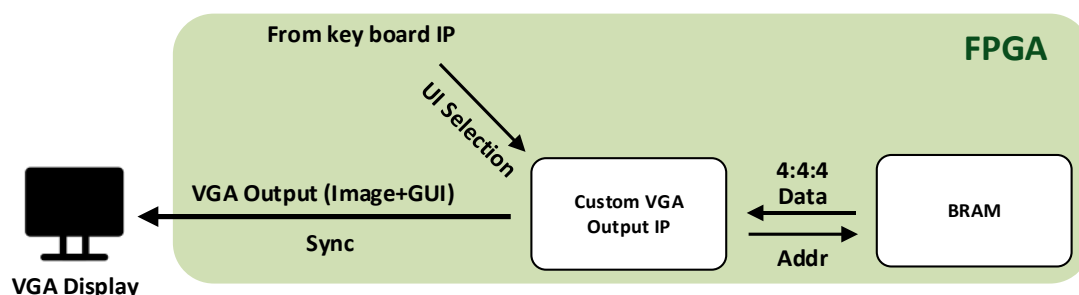


Fig. 11 Block diagram of interfacing between FPGA and VGA display

The VGA output IP will read 4:4:4 RGB data from BRAM. Combined with the control from keyboard, it will display the image data store in BRAM plus the GUI controlled by keyboard IP. Sections below will illustrate the VGA output mechanism and GUI respectively.

4.4.1 VGA output IP

The VGA output IP continuously read the frame data store in BRAM. Fig. 12 below shows the block diagram of the VGA output IP.

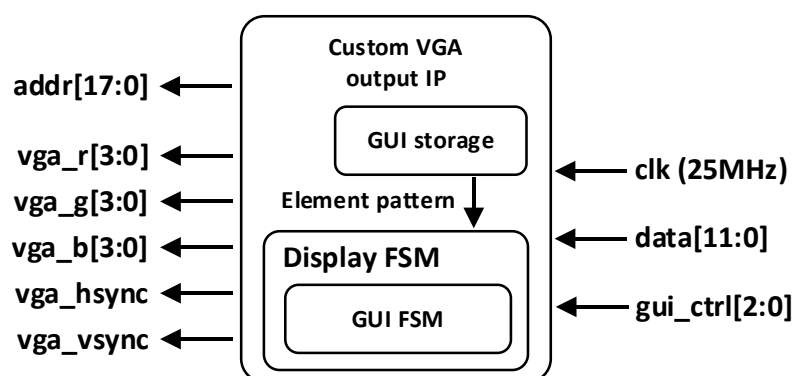


Fig. 12 Custom VGA output IP block diagram

The VGA output IP is driven by a 25MHz clock. It read a 12-bit data from BRAM by giving a corresponding 18-bits address that represents the pixel location. A 3-bit gui_ctrl will control the output GUI pattern.

Inside the IP, the display FSM will continuously send address based on the pixel location and accept the data from BRAM. Combined with the GUI element needed to be displayed, the display FSM

will output each color channel in 4-bit. It will also output corresponding vga_hsync and vga_vsync as described in Fig. 13 below, where a necessary delayed is required between each pixel and line transferred.

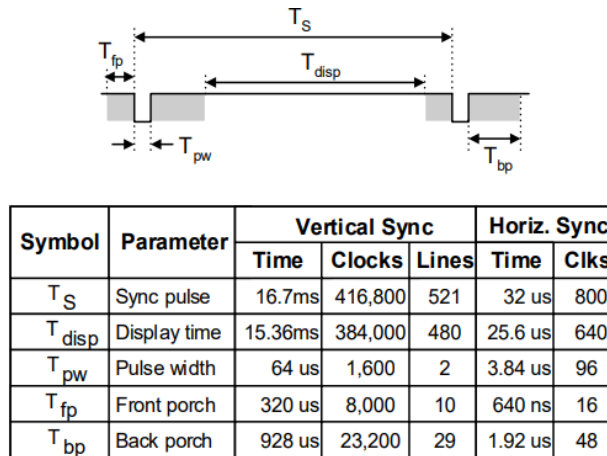


Fig. 13 VGA sync signal requirements [2]

4.4.2 GUI

The GUI block itself consists of two blocks as shown in Fig. 14 where they are used to control and update GUI elements respectively.

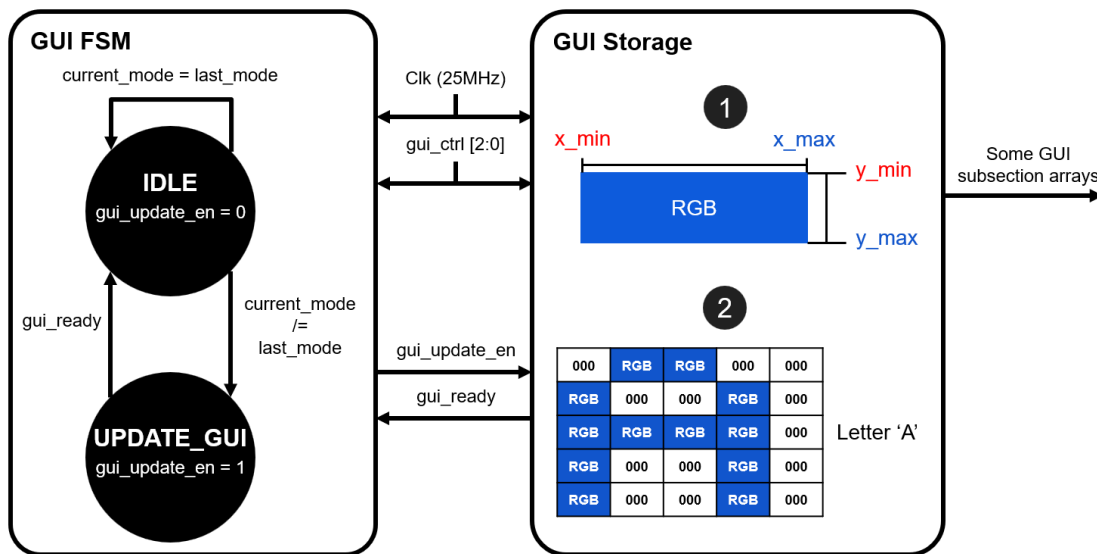


Fig. 14 Custom GUI IP block diagram

The GUI FSM block mainly contains an FSM with two states. The default state is the “IDLE” state which constantly checks for the change of current_mode and last_mode. These two registers store keyboard values read from gui_ctrl signal in the current clock cycle and one before. If a change is detected, the FSM will move to the “UPDATE_GUI” state and flags the gui_update_en signal. This signal is sent to the GUI Storage block to update the GUI elements to be displayed. Upon completion, the GUI Storage block will assert a gui_ready signal back. After receiving the gui_ready signal, the FSM return to the “IDLE” state and pull down the updating flag. The purpose of this block is to

avoid extra work of constantly refreshing the GUI elements and to only update them when new filter is selected.

For GUI storage block, its main purpose is to store, generate, and update the GUI elements. The GUI elements are generated or updated in two methods. The first method uses concatenated if statements to describe the border of certain shapes. This method utilizes runtime execution and does not require many storage resources. Therefore, it is used to generate large but regular shapes which, in the context of this project, are the buttons. However, this method could become very difficult to use with too much if conditions in the case of GUI elements with complex structures. This leads to the second method, which is storing GUI elements are arrays and direct reading from them. This method is used for the update of the letters in the project's GUI. In combination the two, individual elements are combined into larger subsection arrays, which will finally replace the image pixel before display on the VGA monitor.

4.5 Keyboard Interface Unit

The keyboard is connected with the FPGA board through the USB HID port. Instead of directly using HID protocol, this port is used as a PS/2 bus which enables communication with keyboard. The PS/2 bus follows the protocol as shown in Fig. 15 To be more specific, every transaction contains an 11-bit sequence including a start bit, 8 bit data, an odd parity bit, and a stop bit. Following this sequence, the Verilog module is coded to extract the 8-bit data which is used to interpret the key being pressed. In addition, a 100MHz clock is connected to the IP but is divided into 20-30Hz to match keyboard communication.

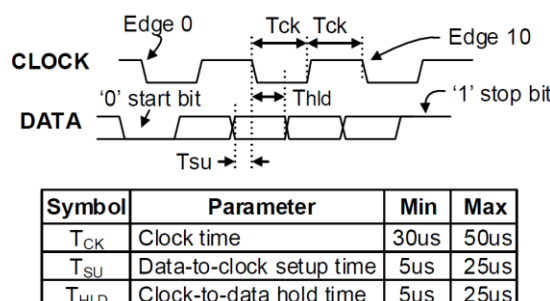


Fig. 15 PS/2 protocol timing diagram [2]

The 8-bit data extracted is in the format of keyboard scan code, where each key has a unique 8 bit code shown in Fig. 16. The project includes 4 modes which used the number keys from “1” to “4” to control. However, in the design process, the team prepared 1 more key, key “5” in cases the need for more filters. As a result, an encoder is created to specifically translate the scan code for these 5 keys into a 3-bit filter selection signal. This signal tells the rest of the system of which filter should be applied and displayed.

ESC 76	F1 05	F2 06	F3 04	F4 0C	F5 03	F6 0B	F7 83	F8 0A	F9 01	F10 09	F11 78	F12 07	
~ 0E	1! 16	2@ 1E	3# 26	4\$ 25	5% 2E	6^ 36	7& 3D	8* 3E	9(46	0) 45	-_ 4E	=+ 55	BackSpace ← 66
TAB 0D	Q 15	W 1D	E 24	R 2D	T 2C	Y 35	U 3C	I 43	O 44	P 4D	[{ 54	]} 5B	\\ 5D
Caps Lock 58	A 1C	S 1B	D 23	F 2B	G 34	H 33	J 3B	K 42	L 4B	:: 4C	'" 52	Enter ↵ 5A	
Shift 12	Z 1Z	X 22	C 21	V 2A	B 32	N 31	M 3A	,< 41	>. 49	/ ? 4A	⬆ 59	Shift 59	
Ctrl 14	Alt 11	Space 29							Alt E0 11	Ctrl E0 14			

Fig. 16 Keyboard scan codes [2]

5 Description of the Design Tree

The complete project is uploaded to GitHub. The design is available in the repository:

https://github.com/HHypoxia/ECE532_final_project.git.

The repository contains all the source code, custom IPs, and our main Vivado project ready to use.

- **/README.me:** It contains the background of our design and instructions how to use the project.
- **/Doc:** This directory contains the final report of our project.
- **Final_report.pdf:** This file is the team's final report
- **/custom:** This directory contains all the custom IPs the team packaged
 - **/greyscale:** This directory is the packaged greyscale IP, with the complete IP project inside as well.
 - **/keyboard_ip:** This directory is the packaged USB PS/2 keyboard IP. It allows users to use keyboard input to control the filter modes.
 - **/mux_ip:** This directory contains the packaged multiplexer IP that selects between the raw camera frame input and the filtered frame input.
 - **/packaged_ip_imageproc:** This directory contains the packaged convolutional filter IP. It has both the Gaussian Blur filter and the Sobel filter.
- **/main:** This directory includes the main Vivado project of our design
- **OV7670.xpr:** This file can directly open the Vivado project of our design.

6 Tips and Tricks

6.1 Handling clock domain crossing (CDC)

When we design the block diagram, there is a common need to cross clock domain when the input and output clock are not same. Here are some examples that can quickly implement CDC in Vivado design.

- FIFO generator: Both input and output are AXI-stream if using AXI protocol. Need to convert both data into AXI-stream standard. However, it is first in and first out, and doesn't need memory mapping, which is easier to use.
- BRAM (or any form of memory): Usually need interface with AXI4 and interconnect. Memory mapping can help to locate the data precisely but may take more resources and need more complex implementations.
- DMA/VDMA: The DMA/VDMA IP also supports CDC transfer. It is helpful when you need high speed data transmission between peripherals but need Microblaze programming and time to properly configure.

6.2 Reduce OV7670 noise

We've seen the output from OV7670 is full of noise and is not as clear as the video example online. It can be caused by the pull up resistor on the sioc and siod is not robust enough. Add extra pull up resistor off board may improve the output noise.

7 Video

The short demonstration video is provided to quickly go over the main project goal, current project design with its hardware demo, and possible future steps. The video can be accessed by the link below:

https://youtu.be/L6g3ST4_0EA

8 Reference

- [1] OmniVision Technologies, Inc., "OV7670/OV7171 CMOS VGA (640x480) CAMERACHIP with OmniPixel Technology," Preliminary Datasheet, Version 1.01, July 8, 2005. [Online]. Available: <https://www.olimex.com/Products/Components/Camera/CAMERA-OV7670/resources/OV7670.pdf>.
- [2] Digilent Inc., *Nexys 4 DDR FPGA Board Reference Manual*, Rev. C, April 2016. [Online]. Available: https://digilent.com/reference/_media/nexys4-ddr:nexys4ddr_rm.pdf.